The Hidden Volume Stops Being a Rendering Primitive

Decoupling interior occupancy from splatted primitives for exact cutting and physical simulation

Anonymous

Preprint

Abstract

An object reconstructed from photographs has a surface and nothing behind it, and methods that fill the hidden interior do so with the same primitives that render it. We measure what that costs. Sweeping a Gaussian's support over four decades, both failure modes are real and neither is escapable: 16 to 45% of a cut section is unpainted below ρ = 0.01, and blending across material boundaries rises through ρ = 2. Two published methods sit four or more decades left of the usable window and buy coverage with sheer count instead, 7,959,789 and 1,381,100 primitives for one watermelon. Replacing the interior with a two-level cube occupancy and giving the boundary its own dual-grid surface holds the same object in 1,065,856 cells, and the cut face becomes a convex polygon per crossed cell in closed form, carrying no approximation term: its area error falls from 0.97% to 0.09% as the lattice refines while the plane residual stays at machine epsilon. Coverage separates the representations only under resolution: at 2048 px every splatted model thins out, ours included, while the same lattice drawn as a volume holds 0.07% across three octaves. The lattice is reached either by quantising a scan or by generating topology from a field, and nothing downstream can tell which.

orange

watermelon

doughnut

Three planes, eight pieces, drawn apart and back. Each object is a cube volume, drawn as the faces of its own cells. No dual grid and no Gaussian anywhere in the picture: `multicut_gif.py` takes the six cube-face offsets and nothing else from the surface code, so what is on screen is the blocky boundary the lattice actually has. The pieces come out of connected-component labelling on the lattice; the colour on every exposed face is read from the volume that was already there, and no face is tinted. On the orange, 850,822 solid cells refine to 1,159,634 leaves in the bands and the pieces range from 127,115 to 174,231 leaves.

1 Introduction

A reconstruction of a real object knows only its outside. Photographs see a surface, so a model fitted to photographs has a surface and nothing behind it, and the moment such an object is cut open the cut reveals an empty shell. That is a problem for any application in which objects are handled rather than merely viewed, such as surgery rehearsal, food and material simulation and games in which things break. It is the problem FruitNinja addresses by generating an interior that is plausible rather than measured, supervising it with depth-conditioned diffusion on cross-sections.

FruitNinja generates an object's interior so that cutting it open reveals something plausible, and stores it as a cloud of small opaque Gaussians. Putting the whole object on a voxel lattice with one Gaussian per occupied cell makes cutting answerable by connected-component labelling, and still leaves a rendering primitive in every cell. This paper asks what those interior primitives are for.

They are doing two jobs. As *volume* they answer whether space is occupied, which cells are connected, which piece a point is in, and what a colliding body has hit. As *image* they are splatted, blended and composited. Filling a hidden volume with a rendering primitive is paid for in count. FruitNinja's released watermelon holds 7,959,789 primitives at a million per unit volume, each with a support scale of 1.11×10^-7 against a nearest-neighbour spacing of 0.0036. Those are points rather than blobs, and they overlap nothing. Making each primitive smaller does not reduce how many are needed.

The resolution is to stop asking one primitive for both jobs. A cube's support *is* its cell, so a solid occupancy renders solid at any resolution with nothing to tune, and a dual-grid surface has an explicit polygonal boundary, so coverage comes from triangles rather than from primitives dense enough to hide the gaps between them.

Three things follow. **The volume becomes solid**, because a cube's support is its cell and a gap is therefore a hole rather than a coverage parameter. **The cut becomes exact**, because a plane through a cube is a polygon in closed form rather than a plane approximated by primitives. **The shell becomes a parameter**, because route 2 builds the lattice from a field, so the skin band is a number given to the builder, and that number decides whether an MPM solver can resolve a stiffness contrast at all.

The two routes are how that separation is reached. A scanned object is quantised onto the lattice; an object nobody scanned has its topology generated from a signed-distance field and its skin projected from six rendered views. Neither needs a fine-tuned generative model, which is what makes the comparisons here comparisons rather than assertions.

1.1 Contributions

- **The cost of a rendering primitive in a hidden volume, measured.** Sweeping a Gaussian's support over four decades on a fixed primitive set, both branches of the trade are real and neither is escapable: 16 to 45% of a section is unpainted at ρ ≤ 0.01, and boundary blending rises through ρ = 2. Between them is a window near ρ ≈ 0.25 to 0.5. Both published methods sit four or more decades to the left of it and buy coverage with count instead. This trade is asserted in the work we build on and has not previously been measured.
- **A volume that covers at any resolution, with nothing to set.** Swept over image size on one object, every splatted representation thins out as pixels shrink, ours included: by 2048 px GaussianFluent's section is 38.7% background, FruitNinja's 2.4% and our own splatted lattice's 3.3%. The same lattice drawn as a cube volume moves from 0.04% to 0.07% across three octaves, because a cell's support is the cell.

- **A cut face with no approximation term.** The visible face of a cut is a convex polygon per crossed cell, intersected with the cell's twelve edges in closed form. The construction is standard; what is new is that it becomes available to a photogrammetric asset at all, since a splatted interior has no volumetric discretisation to cut. Demonstrated rather than asserted: the area error falls from 0.97% to 0.09% as the lattice refines while the plane residual stays at machine epsilon on an oblique off-grid plane, so the only error is the silhouette.
- **Less to hold the interior, with the accounting stated.** On the same watermelon, counting the sparse index and the exterior surface this method needs, 62.6 MiB generated or 163.5 quantised against FruitNinja's 425.1 and GaussianFluent's 310.8. At matched fields, with GaussianFluent's higher-band spherical harmonics removed since this representation has no view-dependent appearance to charge for, its file is 73.8 MiB, so route 2 is 1.18× smaller and route 1 2.6× larger. The evaluation is against uncompressed baselines by design, because the quantity in dispute is the intrinsic density of a representation rather than the efficiency of a codec applied to it. Compression is orthogonal and available to both sides: a structured lattice with implicit addressing is directly amenable to entropy coding and octree or Z-order compaction, which neither changes the primitive counts above nor the closed-form geometry that follows from them.
- **The skin band as a build parameter, and a criterion for it.** Because route 2 builds the lattice from a field, the shell thickness is an argument rather than a consequence of where a reconstruction put its primitives, and we give $\tau_s = (t_{shell}/L)/\Delta x$ as a prior criterion, computable before the first timestep, for whether an MPM transfer can resolve a stiffness contrast at all. Two grids appear in such a pipeline and they are not interchangeable: the filling grid that decides where particles are seeded, and the background grid Δx through which momentum is transferred, which is the one a feature must span. Evaluated against the transfer grid, every build in our sweep lies at τ_s between 0.22 and 0.65, below the support width of the transfer kernel, and the solver's response is correspondingly insensitive to the band. The criterion is the contribution and it is what a builder can act on; section 4.3 reports the sweep it predicts.
- **One lattice, two ways to populate it, and no downstream branch.** A scan is quantised; an object nobody scanned has its topology generated from a field and its skin projected from six rendered views. Cutting, surface conversion, physics and material decomposition read occupancy, adjacency and one feature per cell, and cannot tell which route produced them. Removing object-specific photographs moves either route by about a point of FID at fixed code and budget.

2 Related work

Interior generation. FruitNinja [3] supervises an object's interior with depth-conditioned diffusion on cross-sections and stores it as opaque atomic Gaussians. We keep that supervision unchanged, since the contribution here is not a new way to invent an interior, and change what holds it. The lattice it is quantised onto, and the baseline every number here is measured against, is a Gaussian model quantised as it stands; the representations it descends from are Gaussian splatting [1] and radiance fields [2]. Solid texture synthesis [29] asked the same question before diffusion existed, and InFusion [30] and InnerGS [31] answer it for scenes rather than for objects that are cut.

Surface representations for generated 3D. TRELLIS.2's O-Voxel [10] is a field-free sparse voxel surface: a Flexible Dual Grid in which every active voxel carries a dual vertex and grid-edge intersection flags, handling open, non-manifold and fully enclosed internal surfaces. We use its official encoder without reimplementing it, for the cut face and then for the whole exterior.

Dual contouring. The dual vertex is a quadric's minimiser [5], and quadrics are additive over their planes. That is what lets an adaptive grid be built by collapsing, and the collapse's error be read off the same quadric that placed the vertex. Marching cubes [4] is the alternative that places vertices on edges instead, and cannot represent a feature inside a cell; quadric error metrics [8] are where the additivity comes from, and dual marching cubes [6] and feature-sensitive extraction [7] are the two nearest relatives of what is used here.

Physics on primitives. PhysGaussian [18] simulates Gaussians with MPM. We keep the existing particle set and give it piece identity, so subdividing a cut band multiplies leaves and not particles. The solver is MPM [19, 20] in the moving-least-squares form [21], and the surface follows it by the same weighted-sum binding that skinning [24] and embedded deformation [25] use, which is why an affine motion is followed exactly.

Mixed materials and fracture. GaussianFluent [23] is the nearest current work and reaches the same two obstacles from the other side: it densifies an interior with generative guidance and then simulates brittle fracture with a continuum-damage MPM, handling mixed-material objects in one solver. Where it makes the interior denser so that a Gaussian representation can carry it, this work removes the rendering primitive from the interior entirely, so the comparison worth drawing is not which fractures better but what each pays to hold a volume. The material question we reach, whether a shell survives the grid transfer, is one any mixed-material MPM has to answer.

Structured Gaussians. Attaching primitives to a structure rather than letting them float is the line this work sits in: anchors on a sparse voxel grid [12], a dense structured cube [13], an octree with level of detail [14], and before them the octree and voxel grids that accelerated radiance fields [15, 16] and the hash encoding that replaced them [17]. What those share is that the structure serves the renderer. Here it serves the volume, and the renderer is given its own representation.

Compressing a Gaussian model. Every storage figure here counts fields at float32 on both sides, which is a file format rather than a representation. The literature that closes that gap is not small: sensitivity-aware vector quantisation [40], learnable masking with residual quantisation [41], spherical-harmonic distillation [42], hash-grid context modelling [43], and putting Gaussians on a grid precisely to make them compressible [44], surveyed in [45]. Reported reductions run from fifteen-fold to seventy-five, and primitive count itself can be cut an order of magnitude at equal quality [46, 47]. We do not run any of them, and section 4.2.1 states what that leaves owed.

Cutting a discretisation. Keeping a coarse volumetric discretisation and representing the cut surface analytically inside the cells

it crosses is two decades old in graphics: the virtual node algorithm duplicates a cut element so each copy carries one side [48], arbitrary cutting of deformable tetrahedra produces exact polyhedral fragments [49], and extended finite elements formulate the same idea as enrichment [50], with the surgical-simulation line starting earlier [51]. What has not been available before is any of this on a photogrammetric radiance-field asset, because such an asset has no volumetric discretisation to cut. Fracture by continuum damage [52] is the alternative separation mechanism GaussianFluent uses.

Sub-pixel primitives and antialiasing. A primitive whose screen-space footprint falls below the sampling rate disintegrates as resolution rises, and the fix is a minimum-scale filter: Mip-Splatting caps the extent at the sampling rate for exactly this reason [53], analytic integration does it differently [54], and the framing that a primitive is a reconstruction filter which must match the sampling rate is older still [55]. The released models we measure carry no such filter. Section 4.2.2's claim is therefore that a cube needs no minimum-scale parameter, not that a Gaussian cannot be given one.

Evaluation. FID [32] and KID [33] against photographs, and CLIPScore [34] for whether the picture reads as the thing. Both distribution metrics are used at sample sizes far below where they are reliable [35, 36], which the results section states rather than works around.

3 Method

3.2 Overview

Everything below is one path, and every object takes it. What differs between two objects is a parameter file, never a branch in the code. The figure reads top to bottom in five bands. An object arrives as one of two things (A), reaches the lattice by the route that thing allows (B), and from band C onward there is one structure and no route: the two-level occupancy, its face adjacency and one learned feature per cell. That structure carries two representations (D), the cube hierarchy that answers where material is and the dual grid that draws where it ends, and three families of operation read them (E). Cutting, drawing and physics are separated in the figure because they ask different questions, but they share one property worth stating early: each is decided at the resolution of a cell, so each is wrong in the same place, and each is corrected by stating the geometry exactly where the discretisation cannot. The numbers in parentheses are the equations that define each step.

3.3 Which route an object takes

The two routes are not alternatives to be chosen on preference; each answers a different question about what exists before the pipeline starts.

what you have	route	why
A reconstruction of a real object, with an interior already generated into it	**Route 1**, quantise	The interior exists and is the thing to preserve. Quantisation keeps it and pays only the repair cost of a scan's voids: closing adds 11.7% of cells on the orange and takes the uncut object from 561 components to one.
A shape with no reconstruction, or one whose shell thickness matters to a solver	**Route 2**, generate	The topology comes from a field, so it is solid by construction (closing adds 0.1%) and the skin band β_s is an argument rather than a consequence of where a reconstruction put its primitives. This is the only route under which τ_s can be built for.
Either	it does not matter	Cutting, the surface conversion, the physics and the material field read occupancy, adjacency and one feature per cell, and cannot tell which route produced them.

Route 2 additionally needs six views to paint the skin. Where those come from is a separate question from what the route does with them: of the objects here only one had no reconstruction to begin with, and for the other two the field and the views are derived from the released model, so those two demonstrate that the route reaches the same lattice rather than that it can do without a scan.

3.4 The two-level lattice

One definition serves the whole page. A cell is an axis-aligned cube; its index at level ℓ is a floor division and its spacing halves with the level,

$$h_\ell = frach_c2\hat{}\ell,\ k_\ell(x) = \lfloor fracx - oh_\ell \rfloor,\ o = min_i\ x_i - 1/2\ h_f \tag{1}$$

$$V = V_0 \cup V_1,\ V_1 = k_1 : dist(x_k_1, \partial\Omega) < \beta\ h_f\ ,\ V_0 \cap parent(V_1) = \varnothing \tag{2}$$

$$A = (k, k') : \| k - k' \|_1 = 1 \cup (k_0, k_1) : k_0 = k_1 \gg 1 \tag{3}$$

3.5 Solidity of a quantised occupancy

Route 1 reaches the lattice by quantisation. Every primitive of the source model is sent to the cell that contains it, and the set of cells that received at least one is the occupancy,

$$k(mathbfp) = \lfloor fracmathbfp - oh_\ell \rfloor,\ O = k(mathbfp_i) : i = 1\ dots\ N \tag{4}$$

The half-cell offset in equation (1) is written with h_f, which is what makes equation (3)'s parent relation hold with one shared origin. Two scripts offset by tfrac12 h_c instead when they only ever index at the coarse level, where either choice puts a cell centre in the middle of its own cell; the measurements are unaffected and the inconsistency is noted here rather than left for a reader to find. h_f = h_1 throughout, the fine spacing, and h_c = h_0 the coarse one; the pipeline figure writes them h_e and h_0.

With the origin o of equation (1) offset by half a cell. The offset is not cosmetic. A generated lattice puts its cells at (k + 1/2)h, so flooring from the minimum rather than from half a cell below it lands every cell centre exactly on a cell boundary and lets floating point decide the side: measured on these lattices it loses 49% of the cells.

A quantised occupancy is not solid. Photographs see a surface, so the cells behind it were never occupied by anything and O has interior voids. Closing removes the gaps between neighbouring shells and a flood fill from outside removes the rest,

$$O\hat{}bullet = \complement\, mathcalC_\partial\ (\ \complement\ (\ O \oplus B \ominus B\ )\ ) \tag{5}$$

where B is the one-cell structuring element and mathcalC_∂ is the component of the complement that reaches the bounding box. What is left is inside by construction rather than by a threshold. The count of connected components is the read-out.

object	coarse cells	pieces before	pieces after
doughnut	517,166 → 522,753 (+1.1%)	1	**1**
orange	759,353 → 848,021 (+11.7%)	561	**1**
watermelon	553,043 → 561,805 (+1.6%)	1	**1**

A sponge falls into many connected components and a solid into one, so the component count of the *uncut* object is a direct read-out of whether the occupancy is solid, and it says these are holes rather than fragments. A piece count means nothing until this has run, so the cutting code runs it rather than assuming it.

3.6 Populating the lattice

Everything after this section is shared. An object reaches the same two-level lattice by one of two paths, and which one it took is not recoverable downstream: cutting, the surface conversion, the physics and the material field all read a lattice and none of them asks where it came from. The two paths are how the lattice gets populated rather than the claim being made; section 4.1 measures the independence and section 4.2 the claims that rest on the lattice itself.

Each of its steps replaces an inference with something exact.

step	what it does	what it replaces
topology	cells inside a signed-distance field, at two spacings	quantising a reconstruction, then repairing the holes it left
normals	the gradient of that same field	the direction from the centroid, or a smoothed occupancy gradient
six views	renders of the source model, cropped to the silhouette	images generated by diffusion, one per face
skinning	projection through the camera that made each view, blended by incidence	a saturating map from a cell's direction to a reference radius

What the route needs is a field and six views, and where those come from is a separate question from what the route does with them. Of the three objects here only the doughnut had no reconstruction to begin with; for the orange and the watermelon the field and the six views are derived from the released model, so those two demonstrate that the route reaches the same lattice and not that it can do without a scan. Section 5 states what that leaves unproven.

Route 2 does not quantise anything. The topology comes from a signed-distance field φ evaluated at the cell centres, and the same field gives the normal and the skin band, whose width β is an argument rather than a consequence of where a reconstruction put its primitives,

$$V_0 = k : \varphi(x_k) < 0 \, , \; S = k : -\beta \, h_f < \varphi(x_k) < 0 \, , \; n_k = \frac{\nabla \varphi(x_k)}{\| \nabla \varphi(x_k) \|} \tag{6}$$

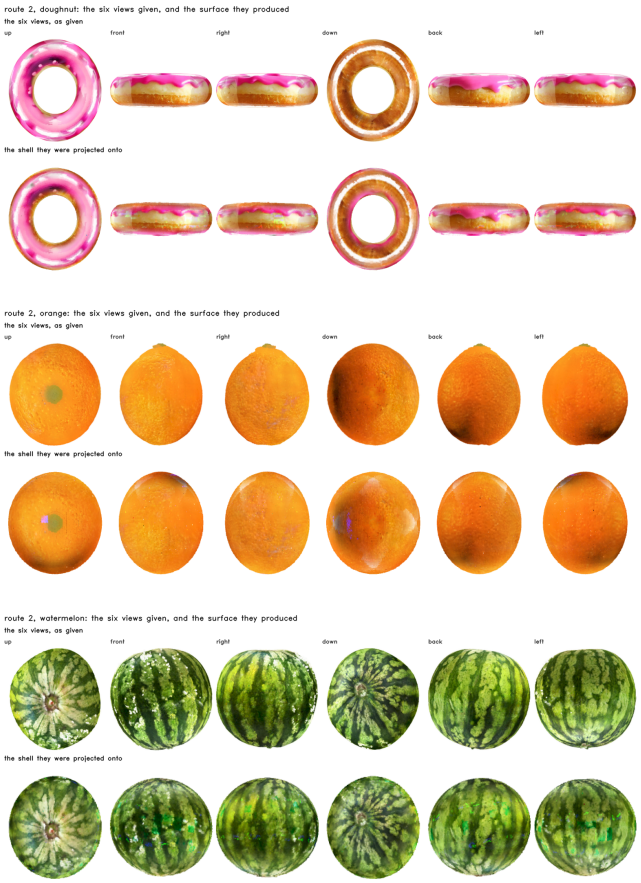
Colour reaches the skin cells by projection, by the standard construction: incidence-weighted blending of registered views is view-dependent texture mapping [37, 38], and iterative six-view projective texturing of a generated shape is what TEXTure [28] does with diffusion images. What differs here is only the source of the views.

In detail: Each of the six views I_i has the camera π_i that rendered it and a direction d_i, so a skin cell is looked up in every view that sees it and the views are averaged by how squarely each faces the cell,

$$c_k = \frac{\Sigma_{i=1}^{6} w_{ik} \, I_i(\pi_i(x_k))}{\Sigma_{i=1}^{6} w_{ik}}, \; w_{ik} = \mathbb{1}[k \in S, \text{visible in } i] \, \max(n_k \cdot d_i, 0)^{\gamma}, \; \gamma = 4 \tag{7}$$

Nothing here is inferred. The camera is known because it produced the view, so there is no saturating map from a cell's direction to a reference radius and no rim light to correct for.

Solid by construction is the first payoff. A cell is inside when the field says so, so there is no scan to recover, no radius to infer and no closing to repair: closing the generated occupancy adds 0.1% of its cells against 11.7% for the quantised orange, and the uncut object is one piece rather than 561.



The four side views in the lower row are displayed with the vertical axis inverted; the lattice is not. The doughnut's icing cells sit at +0.075 along the configured upward axis and its crust cells at −0.034. No measurement here reads from these renders. For each object, the six references above and the skinned generated topology below, cropped to their own silhouettes at one scale so texture can be compared with texture. 100% of every shell is covered and each face takes 16.6 to 16.8% of it, against 48.3% and 3.7% for the two extremes on a quantised lattice.

3.7 Cutting: discrete topology, analytic boundary

A cut arrives as a plane. Cells it crosses are subdivided until h_ℓ ≤ h_target, leaves are joined to their face neighbours, edges the plane separates are dropped, and connected components are the pieces. Written out: a leaf takes the sign of the plane at its centre, an adjacency survives when its two leaves agree, and a piece is a class of the equivalence relation that generates,

$$q(k) = \operatorname{sgn}(n \cdot x_k + d), \; \mathcal{E} = \{(k,k') \in A : q(k) = q(k')\}, \quad \mathcal{P} = V / \sim_{\mathcal{E}} \tag{8}$$

$$|n \cdot x + d| \le 1/2\, h\, \Sigma_a\, |n_a| \qquad (9)$$

A cut here is an unbounded plane, and that is a property of the representation rather than of the implementation. A piece is an equivalence class of face adjacency under a global sign code, one sign per plane, so the cut has no extent: an incision that enters, travels and stops has no expression, because it would need the same pair of cells severed inside the incision's footprint and adjacent beyond its end. A curved cut approximated by Q planes generates up to 2^Q sign regions and shatters the object rather than cutting along a curve. Peeling, tearing and freehand surgical cutting are therefore outside this formulation. The construction that extends to them is known and fits this lattice almost unchanged: duplicate the crossed cell rather than the geometry and let each copy carry one side of an analytic cut surface [48], which is the virtual node algorithm, with the MPM counterpart carrying the discontinuity as two velocity fields either side of a crack.

Two degeneracies have to be handled for the closed-form face to be stable, and both are handled by construction rather than by a tolerance. When the plane passes exactly through a cell vertex, the edge parameter of equation (10) evaluates to 0 or 1 on the two edges meeting there and the same point is emitted twice; the merge that follows collapses vertices within 10^-6h, so the duplicate disappears and the polygon loses a degenerate edge rather than gaining a zero-area triangle. When the plane is coplanar with a cell face, every edge of that face has n·(b-a) = 0 and no intersection is emitted at all: the face is not a crossing, both cells on either side take a definite sign from equation (8), and the cut runs along the boundary already between them. Neither case needs an epsilon on the plane distance, because the sign that decides membership is evaluated at cell centres, which a plane through a vertex or along a face never touches.

Written as an algorithm, with mathcalC the coarse occupancy, Π the plane and K = |c : c ∩ Π neq ∅| the number of crossed cells:

Algorithm 1 Cut, in O(K) after one O(N) preprocess

```
precompute once per object      E_0 ← adjacency(C)                    O(N log N)

cut(C, E_0, Π, h_target)
1   B ← { c ∈ C : |n·x_c + d| ≤ ½h Σ_a|n_a| }      crossed cells, eq. (8)   O(N)*
2   L ← refine(B) until h_ℓ ≤ h_target; L ← L ∪ (C \ B)      leaves          O(K)
3   E ← { (u,v) ∈ E_0 : u,v unrefined } ∪ adjacency(L|_BuaB)   reuse outside the band   O(K)
4   q(ℓ) ← sgn(n·x_ℓ + d)  for ℓ ∈ L          side, eq. (8)                 O(K)
5   E' ← { (u,v) ∈ E : q(u) = q(v) }          drop the crossings            O(|E|)
6   P ← connected_components(L, E')           pieces, eq. (9)               O(|E| α)
7   for c ∈ B do  Π_c ← Poly(c ∩ Π)  by eq. (10)      the cut face           O(K)
8   return  P,  𝒫 = ∪_{c∈B} Π_c
```

* step 1 is a predicate over cells and is the only O(N) term; it is a single vectorised comparison and is not what a cut spends its time on. Steps 2 to 4 and 7 touch only the band. |E| is bounded by 3|L| on a face-adjacency graph.

Step 7 is the geometry the whole construction exists for. For a crossed cell c with vertices V_c and edges E_c, the face is the intersection of a convex body with a half-space boundary, so it is a convex polygon and it is the convex hull of the plane's crossings of the cell's twelve edges,

$$\Pi_c = a_e + t_e\,(b_e - a_e) : e \in E_c,\ t_e \in [0,1]\ ,\ t_e = \mathrm{frac}\text{-}\frac{(n \cdot a_e + d)}{n \cdot (b_e - a_e)} \qquad (10)$$

and the cut surface of the whole object is their union, mathcalP = bigcup_c ∈ mathcalB Π_c. Every vertex of Π_c satisfies n·x + d = 0 by construction rather than by fitting, so mathcalP is planar to whatever the arithmetic leaves and carries no approximation term. Ordering the crossings by angle about their centroid in the plane's own basis gives the polygon; three to six crossings occur for a cube, and the two degenerate cases are handled as described below. The polygons meet edge to edge because every crossed cell is refined to the same depth, so mathcalP is a manifold with boundary wherever the occupancy is.

A cut costs what it touches. Refinement is already local, and so is the rest of it, because a plane changes face adjacency only where it refines: a pair of unrefined leaves is adjacent after the cut exactly when it was adjacent before. So the object's own adjacency is computed once and every cut reuses the pairs whose ends it did not touch, recomputing only the pairs with a refined leaf on either end. That set is the band, which is O(N^2/3) of the volume and measures 4.0 to 10.7% of the cells on the objects here, so the cut is linear in the crossed cells K rather than in the object N. Labelling is then connected components on the surviving graph.

one cut on the orange, 894,304 solid cells	**seconds**
labelling as an interpreted union-find, adjacency over every leaf	13.76
labelling vectorised	5.24
adjacency reused outside the band	**2.91**
the object's own adjacency, once, reused by every later cut	5.00

The three rows produce the same 991,436 leaves, the same 2,925,520 surviving edges and the same two pieces; only the work differs. Checked on every synthetic case of section 4.3.7 as well, where the band-local result matches the full pass in piece count and edge count exactly. What remains at 2.91 s is the refinement itself and the component pass, not the adjacency.

Equation (9) is the standard box-plane separating-axis test [39], and it is used twice. A cell of side h centred at x is cut by the plane exactly when its centre lies within half the cell's diagonal extent along n, which is the right-hand side; that decides which cells are refined. It also bounds how far a point inside a cell can be from the plane, which is what section 4.3.5 uses to explain why occupancy alone claims material on both sides of a cut and why one sign test removes exactly that band and nothing else.

Adjacency is arithmetic too. A leaf's neighbour is at its own level or coarser (were it finer, the finer leaves on the shared face would find this one), so the coarser candidate is the neighbour's coordinate shifted right, which is floor division and therefore the cell containing it. With Q planes a leaf carries a *side code*, the Q signs at its centre, and adjacency survives only between equal codes.

Topology is voxel-approximate by construction; the visible face must not be, or it reads as a staircase. Per cut leaf we intersect the plane with the twelve edges, order the points around their centroid in the plane's own basis, and fan them: a convex polygon of three to six vertices lying on the plane. The residual is what the arithmetic leaves: measured on an axis-aligned plane through the origin it is 0.00e+00 exactly, which is that plane's own accident, and for an oblique plane it is the rounding of a + t(b-a), a small

multiple of the machine epsilon. The claim worth making is not the size of that number but that there is no other error term: no primitive is asked to approximate the plane, so the only departure in a cut is the voxelisation of the silhouette. Every crossed cell is refined to the same depth, so cut polygons meet edge to edge and there are no T-junctions between them.

3.8 The boundary as a dual grid

Converting only the cut face would require a second renderer and a per-pixel visibility rule between it and the first; converting the whole boundary removes both. It is also necessary rather than convenient: a Gaussian shell covers only 7 to 20% of the frame at alpha above one half, so at the silhouette a cut face shows through from the side it should be hidden on.

Converting only the cut face and leaving the photographed outside as Gaussians is what makes two renderers, a depth convention and a compositing step necessary. Converting the exterior as well removes the seam instead of managing it. The occupancy already holds the volume, so the exterior *is* its boundary, every face of an occupied cell whose neighbour is empty, and the same dual-grid encoder that handles the cut face takes the staircase off it.

3.8.1 Adaptive collapse

What follows is Ju et al.'s adaptive dual contouring [5] applied to the Flexible Dual Grid: summing children's quadrics bottom-up and collapsing where the residual is small is their construction, and the additivity that makes it work is Garland and Heckbert's [8]. The only thing new here is expressing the tolerance as a length rather than as a raw residual, which is what makes one τ_c comparable across objects and levels.

A dual grid stores one node per active surface voxel, so it costs O(A/h^2): a plane costs as much as a crumpled sheet of the same extent. That is fixable inside the algorithm, because the dual vertex is already the minimiser of a quadric. For supporting planes (n_i, d_i),

$$Q(x) = \Sigma_i (n_i^{\wedge T} x - d_i)^2 = x^{\wedge T} A x - 2 b^{\wedge T} x + c,\ A = \Sigma_i n_i n_i^{\wedge T},\ b = \Sigma_i d_i n_i,\ c = \Sigma_i d_i^2 \quad (11)$$

so a quadric is ten numbers. The property everything rests on is that it is additive over the set of planes, Q_parent = Σ Q_child, exactly and with no reference to the mesh the children came from. The grid therefore collapses bottom-up: merge a 2×2×2 block, add the quadrics, solve Ax = b, and read the residual r = c - b^Tx, the sum of squared distances from that one vertex to every plane the block covers. The tolerance is a length rather than a residual,

The collapse is a constrained optimisation and is worth writing as one. A block of cells carries the planes its children saw; collapsing it replaces their dual vertices with a single v^* placed where those planes agree best, subject to the surface remaining the boundary of the same solid:

$$v^* = \operatorname{argmin}_v \Sigma_p \in \mathcal{H}(c) (n_p^{\wedge T} v - d_p)^2 \text{ s.t. } \chi(\mathcal{M}) = \chi(\mathcal{M}_0),\ \partial \mathcal{M} = \partial \mathcal{M}_0 \quad (12)$$

The objective is the quadric of equation (11), additive over the planes a block covers, so the sum is ten numbers added per child and the minimiser is one 3×3 solve. The constraint is what keeps a collapse changing the resolution rather than the object: the Euler characteristic and the boundary curve set are held at their uncollapsed values. That is enforced structurally, not by a penalty. A dual grid's connectivity comes from the occupancy and not from where its vertices sit, so a collapse merges vertices and never edges or faces, and a merge is refused when it would identify two vertices the occupancy does not make adjacent, which is the only move that could change chi. The same fact gives manifoldness where a cut face meets the exterior: both are indexed by the same cells, and the seam is exactly the boundary cells the plane crosses.

$$\sqrt{(r / w)} \le \tau_c h,\ w = \text{planes the block covers}, \quad (13)$$

τ_c swept from 0 to 1 and back on the orange's exterior. 421,138 nodes at τ_c=0 and 149,031 at τ_c=1; the surface is the same orange throughout and the rim steps visibly at the end, which is what a coarser node is.

3.8.2 Collapse tolerance against surface error

The collapse has one parameter, τ_c in equation (12), and what it buys has to be read against what it costs. Sweeping it on the orange's own dual grid gives both at once: the node count that survives, and how far the surface moved to reach it. The distances are to the uncollapsed grid and not to an analytic surface, so this is a self-consistency measure of what the collapse discards rather than a surface-error measure against ground truth; the identity row reproduces the uncollapsed mesh, which is what makes it the reference. All distances are in units of h.

τ_c	nodes	of uniform	node → surface	surface → node	symmetric	95th	triangles
−1 (identity)	442,253	100.0%	0.000	0.000	0.000	0.000	934,520
0.10	315,661	71.4%	0.024	0.262	0.163 h	1.000	n/a
0.35	280,491	63.4%	0.036	0.328	0.215	1.000	614,129
0.50	215,304	**48.7%**	0.072	0.468	0.338	1.209	474,742
1.00	149,031	**33.7%**	0.093	0.712	0.556	1.637	326,341

The doughnut's 530,958 nodes fall to 197,017 at τ_c = 0.5 and 143,247 at τ_c = 1.0. The identity row is the check that the rest means anything: an adaptive dual grid has no uniform mesh to hand back to the library, so the faces are rebuilt by resolving each crossed fine edge's four voxels to the nodes that survived, and with nothing merged that reproduces the library's own mesh to four parts in a million: 934,520 triangles, area 15.22346 against 15.22352, and the same 12,606 boundary and 41,947 non-manifold edges its mesh already has.

3.9 Physics on the lattice: pieces, materials, collision

The O-Voxel patch is the visible boundary and takes no part in the physics; the cube hierarchy is the physical volume and answers every question about where material is. Piece identity is a relabelling and not a resampling. The existing particle set stays as it is, and each particle takes the piece of the leaf it falls in, so subdividing a cut band multiplies leaves and not particles. Collision is a floor division and an occupancy test, with one more of each when the coarse cell was refined.

$$\text{occ}_P(x) = [\ \text{leaf}(x) \in P\ ] \wedge [\ q_P (n \cdot x + d) > 0\ ] \quad (14)$$

q_P ∈ +1, -1 is the side of the plane piece P lies on, taken as the sign that the majority of its leaves carry under equation (8);

leaf(x) is the leaf containing x, found by the floor division of equation (4). The sign test is applied only to points inside the band of equation (9), which is where a leaf can straddle the plane at all; a point further from the plane than half a leaf's diagonal extent is inside or outside on the strength of its leaf alone. That restriction is what makes the at-rest result below a consequence rather than a coincidence, and applying the test unconditionally would be a different and stricter collision model.

Near the cut the voxel test claims material up to half a leaf past where the cut is, because a leaf the plane passes through is occupied as a whole. One comparison fixes it: the point must be on the piece's own side of Π. Measured on the current models, two freshly separated halves report material inside each other from occupancy alone; with the plane test, none of them do.

Material comes off the same feature. The decoder's per-cell feature is clustered into K classes, and each class is given a stiffness by an ordering rather than a fit: a class that lies mostly in the skin band is peel whatever its colour, since the shell term dominates; among interior classes the paler one is the structural tissue, pith or albedo or crust, and ranks stiffer. The ordering rather than the absolute value is the claim,

$$u_j = 0.65\,\beta_j + 0.35\,R(L)_j,\; E_j = E_{min} + R(u)_j\,(E_{max} - E_{min}) \tag{15}$$

where β_j is the fraction of class j's cells in the skin band, L_j its mean luminance, R the rank normalised to [0, 1], and E_min = 10^5, E_max = 5×10^6 the ends of the range the ordering is mapped onto. The weights 0.65 and 0.35 are fixed rather than fitted; what is claimed is the ordering they produce, not the values. Each piece then carries a per-particle stiffness field instead of one modulus, and the solver runs once per piece.

Whether the solver can see any of that is a separate question, and it is decided before the first step is taken. MPM carries material through a background grid of spacing Δ x, and a feature thinner than about two grid cells is averaged away in the transfer. So the number that governs a hard shell is the shell's thickness in the solver's own units,

$$\tau_s = t_{shell} / L\Delta x,\; \Delta x = frac{grid\_lim}{n\_grid},\; t_{shell} = P_{90}(\min_{k' \notin S} \| x_k - x_{k'} \|) + \tfrac{1}{2} h_f \tag{16}$$

The percentile in (15) is over the shell cells k ∈ S.

where L is the object's own extent, because the solver normalises the object into its grid box and a world length only reaches Δ x through that scaling.

> **Which grid Δ x refers to decides the whole criterion.** An MPM pipeline carries two, and they serve different purposes. The *filling* grid decides where particles are seeded inside the object and can be as fine as the geometry deserves. The *background* grid is where momentum is transferred, and it is the one a material feature has to span, because a quadratic B-spline weight has support 1.5Δ x either side of a node, so any feature narrower than about 2Δ x contributes to the same nodes as its surroundings and returns from the grid as their average. τ_s must therefore be formed against the transfer grid. In our pipeline the filling grid is Δ x = 0.01 and the transfer grid Δ x ≈ 0.04, a factor of four, and reporting τ_s against the former overstates it by that factor. Against the transfer grid the shells read **0.50** (orange), 0.31 (watermelon) and 0.22 (doughnut), and the band sweep reads 0.29, 0.39, 0.52 and 0.65 at β_s = 3, 5, 7, 9. Every build in reach of this lattice is therefore below the kernel's support width, which is what the solver's insensitivity to the band in section 4.3.4 reflects, and reaching $\tau_s \gtrsim 2$ on these objects needs either a finer transfer grid or a band several times wider than any built here. The criterion tells a builder that before the run rather than after it.

grid_lim and n_grid are the solver's box size and grid resolution. The condition for a stiffness contrast to survive is $\tau_s \gtrsim 2$: MPM carries material through one background grid, and a feature thinner than about two of its cells is averaged into its surroundings by the particle-to-grid and grid-to-particle transfers. We take that threshold from standard MPM practice rather than deriving it, and section 4.3 tests it rather than assuming it. Route 2 makes τ_s something to solve for: t_shell ≈ β h_f, so the band width that reaches a given τ_s is $\beta \geq 2\,L\,\Delta x / h_f$. Section 4.3 measures τ_s on the existing objects, builds a lattice at the predicted β, and reports what the solver does on either side of the threshold.

A cut face and the exterior surface meet at a seam, and the two are produced by different constructions: the face is the planar polygon of equation (10) and the exterior is the dual grid of equation (11). They are joined rather than blended. Both are indexed by the same cells, so the seam is exactly the set of boundary cells the plane crosses, and each contributes one polygon and one dual vertex whose positions already agree to within the cell. The normal is deliberately discontinuous across it: a cut is a crease in the object, and smoothing the normal across the seam would round an edge that physically exists. What is made continuous is colour, because a cell's feature is decoded once and read by both sides, so a face and the rind beside it take their colour from the same cell rather than from two representations that have to be matched. The residual seam artefact is therefore geometric quantisation of the silhouette, which section 4.5.2 measures, and not a shading discontinuity.

A rigid transform per piece is correct only while nothing bends. Each surface vertex is bound to its own piece's k nearest particles with weights fixed at bind time,

$$x_v(t) = \sum_{j \in N_k(v)} w_{vj}\, x_j(t),\; \sum_j w_{vj} = 1 \tag{17}$$

The weights are a smooth kernel over the k nearest particles, whose support is the k-th distance itself, so a vertex in a dense region binds tightly and one in a sparse region reaches further without a length scale being chosen anywhere,

$$w_{vj} = \tilde{w}_{vj} / \sum_{j'} \tilde{w}_{vj'},\; \tilde{w}_{vj} = (1 - r_{vj}^2)_+^3 + \varepsilon,\; r_{vj} = \frac{\| x_v(0) - x_j(0) \|}{\max_{j' \in N_k(v)} \| x_v(0) - x_{j'}(0) \|} \tag{18}$$

ε = 10^-12, which only keeps the outermost weight from being exactly zero. Because the weights sum to one and are fixed at bind time, any affine motion of the particles is followed exactly:

substituting x_j(t) = Ax_j(0) + b into (17) gives x_v(t) = Ax_v(0) + b, which is why the residual below is a statement about bending and not about drift.

4 Experiments

Everything below is a comparison, and every comparison has a picture. The organising axis is the two routes, a lattice quantised from a scan and a lattice generated from a field and skinned from six views, because after the second step nothing downstream can tell which was taken, so any result that separates them is a result about how an object is *reached* rather than about what happens to it afterwards. Comparisons against other work sit on the same footing with one exception, stated in section 4.2: route 1 quantises a released model, and the released watermelon's interior came from a fine-tuned checkpoint, so that lattice inherits one.

the comparison	what it answers	its figure
route 1 against route 2	cost of a generated topology	held-out cuts, transverse and longitudinal
the six views against the shell they made	projection accuracy	three per-object sheets
this work against FruitNinja and GaussianFluent	cost of holding a hidden volume	the same slab, four ways
this work against FruitNinja's released models	the cut a viewer actually sees	the same held-out planes, five rows
occupancy alone against occupancy and the cut plane	whether contact is real	the contact slab at four separations
with photographs against without	dependence on object-specific supervision	the same cuts, both ways
the cube renderer against the rasteriser	representation against renderer	the same cuts, four columns
uniform material against a field	solver-resolvable shell thickness	the section by class and by stiffness

4.2 Comparison with prior work

Two published methods fill an object's interior with Gaussian primitives so that a cut reveals something: FruitNinja [3], which supervises the interior with depth-conditioned diffusion, and GaussianFluent [23], which densifies it under generative guidance and then simulates fracture. They are compared here on four things: what the interior costs to hold, what it looks like on terms none of the three methods chose, what a held-out cut scores through one evaluator, and what each method depends on. The last is the axis. FruitNinja's published interiors come from a diffusion model fine-tuned on the object's own photographs, and that model is not released; GaussianFluent needs no fine-tuned model, and neither does this work, so the reproduction anyone can perform is the one without it.

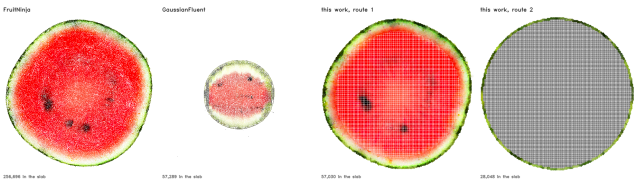
4.2.1 The same object, from the same source file

FruitNinja's released watermelon is redistributed by GaussianFluent [23]: the same 541,266,069 bytes and the same MD5 as the model route 1 quantises. The two representations can therefore be compared on the same object, from the same source file.

watermelon	FruitNinja, as released	this work, route 1	
primitives / cells	7,959,789	1,741,169	**4.6× fewer**
as stored, cells only, index implicit	425.1 MiB	56.5 MiB	7.5× less
with the sparse index the lattice needs	425.1 MiB	76.4 MiB	5.6× less
with the dual-grid exterior as well	**425.1 MiB**	**163.5 MiB**	**2.6× less**
opacity	1.0000 at the median and the 10th percentile	n/a	opaque by construction
support scale σ	1.11e−7	n/a	
spacing h	0.0036	0.0172	
ρ = σ/h	**3.1e−5**	0.26 as splatted; n/a as a cube volume	
primitives per unit volume	1,029,854	137,000	

4.2.2 The interiors compared

Storage says what each costs, not what each produced. Every method has its own frame, scale and renderer, so a comparison of the interiors routes through none of them: for each model, the plane through its own centroid, the primitives within a slab of half-width 1% of its own extent, so 2% thick, projected orthographically, normalised by its own projected radius, opaque and nearest-first, which all three are, at median and tenth-percentile opacity 1.0000.



The same slab through the same fruit, four ways, with every primitive drawn at its own footprint: σ for a Gaussian, h for a cell. GaussianFluent's sits smaller in frame because each model is normalised by its own projected radius and its released file carries more than the melon. No unpainted percentage is printed under the tiles, because a footprint drawing answers what a primitive *covers* and not what a renderer draws; the table below answers the second.

Measured on the images each method's own rasteriser produced, over the same hundred held-out transverse cuts at 512 px, the picture is different and coverage barely separates the released models:

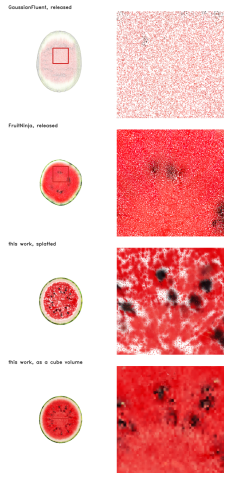
unpainted inside the section, own rasteriser, 100 cuts	watermelon	orange
FruitNinja, released model	0.21%	0.90%
FruitNinja, stock checkpoint	not run	11.40%
GaussianFluent, released model	0.10%	no orange released
this work, route 1	0.17%	0.18%
this work, route 2	**0.00%**	**0.01%**

The one row that is not under a percent is the stock-checkpoint reproduction, at 11.40%, and it is a result rather than an embarrassment: without the fine-tuned model the supervision leaves the interior sparse as well as brown, so the same collapse shows in coverage and in FID.

Two things qualify that table. Every row in it, ours included, is a *Gaussian* render: a route-2 cell carries fourteen numbers and is splatted like any other primitive, at a median ρ of 0.26, which the sweep below shows is inside the window where a Gaussian set pays little for coverage. So this is not yet a comparison between a cube and a Gaussian: at this image size every row is a Gaussian render inside the window where a Gaussian set pays little for coverage, and no claim about a cube's support can rest on it.

And a coverage number at one pixel count says nothing, which is the objection that retired the footprint figure and applies here too. Swept over resolution, on the watermelon, six transverse cuts per cell, with the cube volume rendered by cube lookup rather than by splatting and with supersampling off, so that every row is one sample per pixel. That last point is not incidental: the cube march defaults to two-by-two supersampling, and measuring it that way against Gaussian rows that get one sample gives the cube four times the sample density at the resolution where the argument is made. With supersampling on it reads 0.02 / 0.05 / 0.06% instead of the figures below, which is a small difference and would have been an unfair one. The row above is a hundred cuts at 512 px and these are six at each of three sizes; they are separate experiments and the counts are given so they are not read as one:

unpainted inside the section, watermelon	**256 px**	**1024 px**	**2048 px**
FruitNinja, released model (3DGS)	0.13%	0.39%	2.37%
GaussianFluent, released model (3DGS)	0.05%	2.03%	**38.73%**
this work, route 2, splatted (3DGS)	0.00%	0.45%	3.32%
this work, route 2, as a cube volume	**0.04%**	**0.07%**	**0.07%**



The same held-out plane at 2048 px, each model through its own renderer, with a 300 px crop of the flesh magnified on the right. What the coverage numbers are counting is visible directly: GaussianFluent's section is mostly the background showing between primitives, FruitNinja's is dense but speckled, and this work's own splatted lattice is mottled in the same way because a route-2 cell is a Gaussian like any other. The cube volume is continuous. Its cost is the other thing the crop shows, the cell quantisation, which is the silhouette error of section 4.5.2 seen from inside: a cube covers exactly and it covers at the resolution of a cell.

This is the claim. Coverage from a splatted representation is resolution-dependent for every model including ours: at 256 px they are all under two tenths of a percent and indistinguishable, and by 2048 px GaussianFluent's section is 39% background (32.9 to 49.5 across the six cuts), FruitNinja's 2.4% (1.5 to 3.4) and our own splatted lattice's 3.3% (2.5 to 3.8), against the cube volume's 0.07% (0.06 to 0.09). The cube volume does not move, 0.02% to 0.06% over three octaves, because a cell's support is the cell and a pixel either falls inside an occupied one or does not. What a Gaussian buys with enough primitives at a chosen zoom, a cube has by construction at every zoom, and that follows from what a cell is rather than from a number that happened to come out well. It is measured on one object at three image sizes, on six cuts each, and the other two objects and the intervening sizes are not run.

watermelon	**Fruit-Ninja**	**Gaussian-Fluent**	**this work, route 1**	**route 2**
primitives / cells	**7,959,789**	1,381,100	1,741,169	**1,065,856**
stored, cells only	425.1 MiB	310.8 MiB	56.5 MiB	34.6 MiB
stored, with the sparse index and the exterior surface	**425.1 MiB**	**310.8 MiB**	163.5 MiB	**62.6 MiB**
higher-band SH coeffi-cients	0	45 (76% of its bytes)	0	0
stored at spherical-har-monic degree 0, matched fields	425.1 MiB	**73.8 MiB**	163.5 MiB	62.6 MiB
support σ / cell h	1.11e−7	1.35e−9	0.0172	0.0172
that, against one pixel in the slab figure	47,053× smaller	5,067,762× smaller	3.3 pixels across	3.3 pixels
primitives in the slab	256,696	57,289	51,763	34,587
unpainted at each primit-ive's own footprint (what a primitive covers, not what a renderer draws: it charges a Gaussian its σ and credits a cell its h, and our cells carry Gaussians at σ ≈ 0.26h)	20.5%	46.7%	5.7%	6.1%
unpainted, own rasteriser, 512 px	0.21%	0.10%	0.17%	0.00%
unpainted, own raster-iser, 2048 px	2.37%	**38.73%**	not run	3.32%
the same lattice as a cube volume, 2048 px	n/a	n/a	not run	**0.07%**

4.2.2.1 Then enlarge the Gaussians: what ρ actually costs

The obvious reply to any of this is that a Gaussian's support is a free parameter, so make it bigger. FruitNinja [3] asserts the trade in the form $\rho \ll 1$ leaves holes in the volume and $\rho \gtrsim 1$ blends across material boundaries, and asserting a trade is not the same as showing that no setting of its parameter escapes one. This measures both branches against a reference with no scale parameter at all: a piecewise-constant cube lookup that gives each pixel the colour of the cell containing it, so every cell edge is a step and there is no support to tune. That is not the trilinear reconstruction the renderer uses, which interpolates over eight cell centres and therefore has a one-cell support of its own (supplement, section 4.5.4). Holes are pixels inside the true section the Gaussian render left as background. Blending is how much more the Gaussian render disagrees with that reference *at material boundaries* than it does away from them, since a large Gaussian disagrees with a piecewise-constant reference every-where simply by being smooth, and only the excess is blending.

ρ = σ/h	orange		watermelon		doughnut	
	holes	blending	holes	blending	holes	blending
0.0001	**44.7%**	−0.09	**16.5%**	−0.07	**44.4%**	−0.02
0.01	44.4%	−0.09	16.3%	−0.07	44.2%	−0.02
0.05	37.1%	−0.10	11.8%	−0.06	38.7%	−0.02
0.10	17.5%	−0.09	2.6%	−0.05	n/a	n/a
0.25	**0.0%**	**+0.005**	**0.0%**	**+0.020**	**0.0%**	**−0.009**
0.50	0.0%	+0.028	0.0%	+0.045	0.0%	+0.011
1.00	0.0%	+0.107	0.0%	+0.072	0.0%	+0.030
1.50	0.0%	+0.198	0.0%	+0.089	n/a	n/a
2.00	0.0%	+0.226	0.0%	+0.097	0.0%	+0.074
3.00	0.0%	+0.197	0.0%	+0.105	n/a	n/a

At 1024 px, one plane through the middle of each object, the same plane for every ρ, on this work's own primitive set so that only σ changes. The doughnut's missing cells are values not swept for it rather than failures. Both branches are there and neither is avoidable, but the curve is not the one that assertion implies. The negative blending values at small ρ are the measure reading a render that is mostly holes: with most of the section unpainted, the disagreement away from boundaries exceeds the disagreement at them and the excess goes below zero, so the column only means anything once the holes are gone. The hole branch does not bite gradually below one: it is flat and severe from 10^-4 to 0.05 and then collapses to nothing between 0.10 and 0.25. The blending branch rises through ρ = 2 on all three objects, reaching a mean absolute colour disagreement at material boundaries, beyond what the same render shows away from them, of 0.226 on the orange, 0.097 on the watermelon and 0.074 on the doughnut; on the orange it then falls back to 0.197 at ρ = 3, where the primitives are large enough to be disagreeing everywhere. Between them is a window, roughly $0.25 \le \rho \le 0.5$, where a Gaussian set does pay very little for either, and the models this work trains sit inside it at a median ρ of 0.26.

So the answer to "just make them bigger" is: on a fixed primitive set it works, within a window you have to find, and it is not what the published models do. FruitNinja sits at $\rho = 3\times10^{-5}$, four orders of magnitude to the left of that window, and GaussianFluent's σ is two orders smaller again, though its own nearest-neighbour spacing is not published so no ρ can be formed for it here, where the table says 16 to 45% of a section is unpainted for a primitive set of this size. Their renders are not full of holes because they buy coverage the other way, with 7,959,789 and 1,381,100 primitives against this lattice's 1,741,169 quantised and 1,065,856 generated. The sweep above is on the orange's 877,494, which is a lattice of one particular size; whether an 8-million-primitive set at ρ = 0.25 blends or merely covers is the same experiment on their primitive sets and is not run here. That is the trade this paper is actually about, and it is a trade about count rather than about support: a cube's support is its cell, so it covers exactly, at any resolution, with no window to find and no parameter to set.

4.2.2.2 And under compression

Counting fields at float32 measures a file format rather than a representation, and the obvious objection is that a codec would close the gap. It does not. Every arm below goes through the same appearance-neutral pipeline: constant fields dropped, unobservable fields dropped, positions quantised to sixteen bits of the model's own extent, log-scales and colours to eight, and the result entropy-coded field by field. For the lattice the cell coordinates are Morton-ordered and delta-coded first, which is the octree compaction a structured representation is open to and a free point cloud is not.

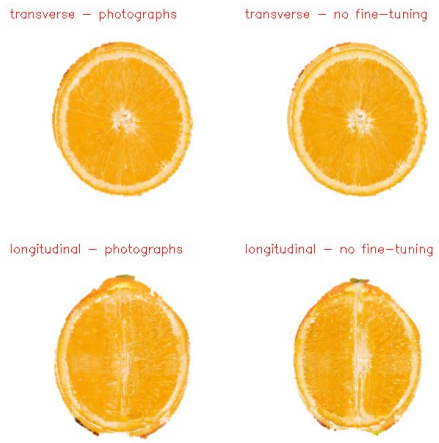
watermelon	as counted	same codec, compressed	ratio
FruitNinja, released	425.1 MiB	58.3 MiB	7.3×
GaussianFluent, released	310.8 MiB	26.5 MiB	11.7×
this work, route 1	76.4 MiB	11.6 MiB	6.6×
this work, route 2	46.8 MiB	**7.0 MiB**	6.6×

The Gaussian models compress harder, which is expected and is the objection stated properly: FruitNinja's opacity has a single value across all 7,959,789 primitives, and its median anisotropy is 1.06, so the quaternion describes the orientation of a sphere. Removing what carries no information takes 425.1 MiB to 58.3 before any entropy coding does its part. The lattice has less of that redundancy to give up and still ends smaller: **3.8× smaller than GaussianFluent and 8.3× smaller than FruitNinja** after every arm has been compressed the same way. The lattice figure is also an upper bound, because the per-cell feature is substituted here by white noise of the same shape, which is the least compressible object of that size; a learned feature can only do better.

4.2.3 Reproduction without a fine-tuned diffusion model

FruitNinja supervises the interior with depth-conditioned diffusion on cross-sections, and its published results use a model fine-tuned on the object's own photographs. That fine-tuning is not released, so the reproduction anyone outside the group can actually perform is the procedure without it: the same base depth2img checkpoint, the same prompt, and nothing fitted to this fruit. Running that arm is one environment variable: removing the photograph directory switches the supervision from photographs back to the diffusion model that FruitNinja specifies.

orange, 100 held-out cuts per family	transverse FID	longitudinal FID
route 1, supervised by the photographs	91.5	222.8
route 2, supervised by the photographs	**74.1**	286.8
no fine-tuning: base diffusion, no photographs	**90.1**	**213.5**
the photographs against each other	46.3	70.2



The same held-out cuts, supervised two ways. Both panels are renders of a trained model, not photographs. Left of each pair: the model supervised by the reference photographs of this orange. Right: the model supervised by a stock depth2img checkpoint and a text prompt, with no photograph of this fruit anywhere in the run, which is FruitNinja's procedure without the fine-tuning it does not release. Top row transverse, bottom longitudinal.

> **What the photographs buy, on route 1, is not measurable here.** Supervising the interior with a stock Stable Diffusion depth2img model and a text prompt, with no photographs of this orange anywhere in the run, scores 90.1 against route 1's 91.5 transversally and 213.5 against its 222.8 longitudinally, on a hundred cuts per family. Those are the same numbers, and 90.1 against 91.5 sits well inside a reference set that scores 46.3 against itself.
>
> Route 2 with the photographs scores 74.1 on the same family, but that is the most in-sample number in the table rather than the best model: route 2's photograph arm was fitted to the six images it is scored against, and the no-photograph arm was not. The two are not comparable and the 16-point gap is not evidence about supervision. The missing cell has now been run, and it closes the square: The claim this work makes is that it does not need a diffusion model *fine-tuned on the object*, which is a different and weaker statement than not needing the photographs at all.

orange, transverse FID ↓	**with the photographs (in-sample)**	**without them (out-of-sample)**
route 1	91.5	90.1
route 2	74.1	**73.2**
FruitNinja, released (out-of-sample)	n/a	102.2

Removing the photographs costs route 1 nothing (91.5 to 90.1) and route 2 nothing (74.1 to 73.2), and both no-photograph arms are scored out-of-sample where both photograph arms are not, so the true comparison is even flatter than the columns suggest. The best out-of-sample number on this object is route 2 with no photograph of the fruit anywhere in its run, at 73.2 against FruitNinja's 102.2. Longitudinally the same arm is 225.4 against route 2's 286.8 and FruitNinja's 269.6.

> **What this does not establish.** The orange's route-2 lattice is built from a field and six views derived from FruitNinja's released orange, whose interior was produced by the fine-tuned checkpoint. So no arm in this table is free of a fine-tuned generative model: what is removed is the object's photographs and any fine-tuning in *our* supervision, not the fine-tuned model that placed the segment structure being quantised. The only object here that never touched a released reconstruction is the doughnut, and the doughnut has no distribution metric because it has one reference image per family. Establishing the claim properly needs an object generated from a field and scored against real photographs, and this paper does not have one.

The interesting question is whether that holds for the method being reproduced. We have the run: FruitNinja's own trainer, its own Gaussian representation and its own filled model, with the stock depth2img checkpoint in place of the fine-tuned one, 200 epochs, scored on the same hundred planes as every other row.

More than the checkpoint differs. Our copy of FruitNinja's trainer supervises sixteen transverse planes where the released schedule supervises fifty, smooths on a 16-cell grid where the released default is 512, and refines every 30 epochs where the released default is 101. So this row is a reproduction under *this work's* training schedule with a stock checkpoint, and the 276-point gap bounds nothing about FruitNinja: it is the sum of removing the fine-tuned model, the schedule change and whatever reproduction gap remains, so we do not read it as a measurement of FruitNinja's dependence. What it does show is the qualitative failure in the figure below, which no budget difference explains: the flesh goes brown and the segment walls become a crust.

orange, the effect of removing the fine-tuned model	**transverse FID ↓**	**longitudinal FID ↓**
FruitNinja, released model (with its fine-tuning)	102.2	269.6
FruitNinja's pipeline, our schedule, stock checkpoint (not attributable to FruitNinja)	**378.7**	**511.6**
this work, route 1 (photographs)	91.5	222.8
this work, no fine-tuning and no photographs	**90.1**	**213.5**
the photographs against each other	46.3	70.2

What the row does support is the comparison inside our own column: this work's two arms, with and without object-specific supervision, are the same code at the same budget. Across columns nothing is bounded. The difference is still visible before any metric is computed: the reproduction's flesh goes brown and its segment walls turn into a crust, because the stock checkpoint has no idea what this particular fruit looks like inside and the Gaussian supervision has nothing else to lean on.

4.2.4 Both methods through the same evaluator

The tables above compare what each representation costs and what each puts inside an object. Neither says which one draws a better cut, and the honest way to ask that is to send every released model through one evaluator: the same held-out planes, the same reference photographs, the same FID, PSNR, SSIM and LPIPS code. Both prior methods release models, so both are run. Every row below is 100 rendered cuts per family, at transverse depths drawn from the middle 30 to 70% of the object and at longitudinal azimuths at least six degrees from any azimuth the trainer walked, with the transverse camera at the pole, where the look-at's roll is given a definite value rather than left to rounding noise, so that no model is framed from inside itself. The reference side is unchanged and small: six photographs for the orange, twenty transverse and twenty-three longitudinal for the watermelon. A

hundred renders against six photographs gives FID an Inception covariance of rank at most five, so every difference in the orange table is inside its own bias term and the ordering is worth more than the gaps.

KID is reported with the standard deviation over fifty subsets that the evaluator computes, and it is the metric to read: it is unbiased where FID at this sample size is dominated by its own bias. Read that way the orange separates less than the FID column suggests, since every arm's KID overlaps FruitNinja's within about two standard deviations except route 2 and the stock-checkpoint arm, and the watermelon separates more, with GaussianFluent's 151±10 clear of everything else. PSNR, SSIM and LPIPS score each render against its *best-matching* reference, chosen by PSNR, which is an upper bound on agreement rather than a paired score, and SSIM is computed on luminance alone. On the watermelon the reference set scores 0.6290 SSIM against itself, below all four methods, so SSIM has no discriminative power there and its column should be read as a null result.

One thing route 1 does not escape, and it is worth stating before the tables: what it quantises is the released model, and for the watermelon that model's interior was produced by the fine-tuned checkpoint. The route-1 watermelon rows therefore inherit the artefact whose absence is this paper's comparison axis, which is part of why they land on top of FruitNinja's own numbers. The claim of independence from a fine-tuned model belongs to route 2, and to route 1 only when what it quantises did not come from one.

> **The orange's evaluation photographs are the orange's training targets.** `objects/orange.conf` sets `REF_H=secref_orraw_hsep` and `EVAL_REF=secref_orraw_hsep`, the same directory, so the two photograph-supervised arms were fitted to the six images they are then scored against. The no-photograph arm, FruitNinja's released model and the stock-checkpoint reproduction were not. The 46.3 reference-against-itself floor is computed on those same targets. Every orange number below is therefore in-sample for two of five rows and out-of-sample for three, and the ordering must not be read as a comparison. The watermelon does not have this problem: its training reference is `secref_wm_h1` and its evaluation set is `data_finetune_images/watermelon/`, which are disjoint. That asymmetry is the mechanism behind a result the earlier text called a puzzle, that every arm here beats FruitNinja on the orange and does not on the watermelon.

FruitNinja's models are in the frame our cameras already use, so they need no alignment. GaussianFluent's watermelon is its own capture in its own frame, and neither its upright axis nor a camera distance is published, so two things were chosen for it: the camera radius, set so that its silhouette fills the same fraction of the frame as ours and FruitNinja's (0.136 against 0.132 and 0.129), and the upright axis, tried as each of the three coordinate axes with the best of the three reported. It is also a different watermelon from the one photographed, which FID tolerates because it compares distributions, and which no per-pixel score really does.

orange, 100 cuts per family	**transverse FID ↓**	**transverse KID ↓**	**longitudinal FID ↓**	**longitudinal KID ↓**	**PSNR ↑**	**SSIM ↑**	**LPIPS ↓**
FruitNinja, released model (fine-tuned on this fruit)	102.2	171±27	269.6	383±90	15.25	0.7561	0.4575
FruitNinja's pipeline, stock checkpoint (our reproduction)	378.7	667±103	511.6	825±79	13.09	0.6461	0.4619
this work, route 1	91.5	142±21	222.8	275±96	16.13	**0.7650**	0.4070
this work, route 2	**74.1**	**101±21**	286.8	382±102	**16.43**	0.7560	**0.4048**
this work, no fine-tuning	90.1	136±27	**213.5**	**298±111**	16.07	0.7632	0.4080
the photographs against each other	46.3	n/a	70.2	n/a	21.24	0.8047	0.1815

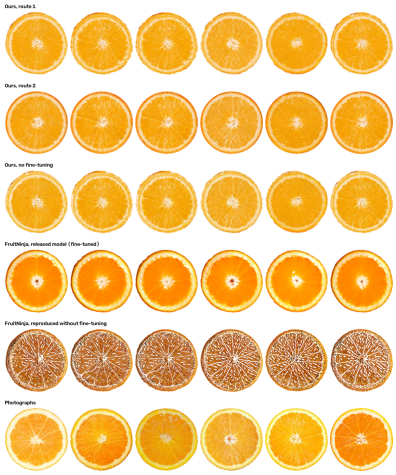
watermelon, 100 cuts per family	**transverse FID ↓**	**transverse KID ↓**	**longitudinal FID ↓**	**longitudinal KID ↓**	**PSNR ↑**	**SSIM ↑**	**LPIPS ↓**
FruitNinja, released model (fine-tuned on this fruit)	182.0	223±15	211.0	244±19	11.32	**0.7383**	0.4651
GaussianFluent, released model (its own watermelon)	**170.2**	**151±10**	**177.5**	**187±18**	**11.61**	0.7119	0.4838
this work, route 1	183.5	240±12	236.7	285±30	11.46	0.7308	0.4625
this work, route 2	180.9	229±14	262.8	342±38	11.51	0.7302	**0.4594**
the photographs against each other	60.4	n/a	69.9	n/a	17.59	0.6290	0.2927

On the orange every arm of this work is ahead of FruitNinja's released model on every column, and two of those arms were fitted to the photographs they are scored against, so that ordering is not a comparison. The two rows that can be compared with FruitNinja's are the no-photograph arm at 90.1 and the stock-checkpoint reproduction at 378.7, both out-of-sample like FruitNinja's 102.2. That narrower comparison is the one we would defend on this object.

On the watermelon the answer goes the other way and should be stated plainly: **GaussianFluent's released model scores best**, 170.2 and 177.5 against our 180.9 and 236.7 and FruitNinja's 182.0 and 211.0. It is a different watermelon from the one

photographed, its cameras were chosen by us, and its upright axis is the best of three we tried while every other row is a single draw, so it is a minimum over configurations against single draws and not a like-for-like win. It is still the best FID in the table and nothing here is served by hiding that. On cost the honest reading is narrower than this paper first claimed. GaussianFluent's file is 310.8 MiB, but 76% of that is degree-1-to-3 spherical harmonics, a view-dependent capability this representation does not have at all. Charged the same fields, at degree 0, its 1,381,100 primitives cost **73.8 MiB** against this work's 62.6 and route 1's 163.5. So against GaussianFluent the storage advantage is 1.18× for route 2 and a 2.6× *disadvantage* for route 1, and the five-fold figure was an accounting of somebody else's spherical harmonics. What does not change is that at 2048 px its own rasteriser leaves 38.7% of its section unpainted where the same lattice drawn as a cube volume leaves 0.07%. The claim this work makes is about what a representation can state about a cut face and what it costs to hold a volume, not about winning a distribution metric at 100 samples against a reference set that scores 60.4 against itself.

These FID values are not the ones in section 4.4.2, which scores six cuts rather than a hundred; the two tables are internally consistent and should not be read across.



The same six held-out transverse planes of the orange, rendered from every model, with the photographs underneath. FruitNinja's released model runs saturated and its pith ring is brighter and thinner than any photograph. The row below it is the same pipeline with the stock checkpoint in place of the fine-tuned one and nothing else changed, which is the reproduction anyone outside the group can perform: the flesh goes brown and the segment walls become a crust. The third row from the top is this work with no photograph of the fruit anywhere in its run, and it is hard to tell from the two rows above it.

The same for the watermelon, with GaussianFluent's own released model in the fourth row. FruitNinja's resolves fewer seeds than ours and softens the ones it keeps. GaussianFluent's is a different fruit, cut in a frame whose upright axis is not published and which we chose in its favour: its rind and pith read thicker than any photograph and its seeds smear along one direction, and it still takes the best FID in the table. The metric is measuring the colour statistics of a slice, not whether the slice is a plane.

4.2.6 The comparison in one table

Each cell is measured on the watermelon by the section named, or marked where the quantity does not exist for that representation. A cut-face residual and a cut-area error need a cut-face construction to measure; a splatted interior draws whatever primitives lie near the plane and has none, which is the difference this paper is about and not a number it can report on their behalf.

dimension	FruitNinja	GaussianFluent	this work	§
primitives or cells	7,959,789	1,381,100	**1,065,856**	4.2.1
stored, all a method needs	425.1 MiB	310.8 MiB	**62.6 MiB**	4.2.1
stored, same codec	58.3 MiB	26.5 MiB	**7.0 MiB**	4.2.2.2
unpainted at 512 px, own renderer	0.21%	0.10%	**0.00%**	4.2.2
unpainted at 2048 px	2.37%	38.73%	**0.07%** as a volume	4.2.2
cut-face plane residual	no cut-face construction	no cut-face construction	**machine ε**	4.5.2
cut-area error against a closed form	not measurable without one	not measurable without one	0.97% → **0.09%**, first order in h	4.5.2
transverse FID, 100 held-out cuts	182.0	**170.2**	180.9	4.2.4
view-dependent appearance	none	45 SH bands	none	4.2.1
slicing and piece labelling	not addressed	damage field	O(K) in crossed cells, 2.9 s	4.5.1

The view-dependence row is a bound rather than a measurement, and it is the one honest way to read the storage column. The decoder here takes a cell's feature and returns a colour; it takes no view direction, so a wet cut face, a specular rind and a translucent flesh are outside what this representation can express, not degraded versions of them. Nothing in this paper measures appearance under moving illumination for that reason. The

storage advantage is therefore partly a capability difference: we do not pay for view-dependent appearance because we do not provide it, and a fair reading of the compressed row is 7.0 MiB for a Lambertian volume against 26.5 MiB for one that also carries a third-order angular field.

Two rows go against this work and are worth reading rather than skipping. GaussianFluent takes the transverse FID, on a different watermelon through cameras we chose for it, and it does so while spending 45 spherical-harmonic bands on view-dependent appearance that this representation cannot express at all. And the slicing row is now an implementation rather than an asymptotic hope: reusing the object's adjacency outside the band takes a cut from 13.8 s to 2.9 s with an identical result. That is still not interactive, and section 3.7 says where the remaining time goes.

4.3 Materials, collision and a shell the solver can resolve

The lattice answers three physical questions with the same structure it uses to answer geometric ones: which material a cell holds, whether two pieces are touching, and whether a stiffness contrast survives the solver's grid transfer. They belong together because they share a failure mode. Each is decided at the resolution of a cell, so each is wrong in the same place, the band where the cut or the shell is thinner than the cell that represents it, and each is fixed by the same move: state the geometry exactly where the discretisation cannot.

4.3.2 Shell thickness in solver grid cells

A material field can change the simulated dynamics and still fail to produce a hard shell around a soft interior, because MPM transfers through one background grid and a shell one to three cells thick does not survive that transfer. That failure has a number, and the number can be worked out before any simulation runs. MPM moves particle quantities to a background grid of spacing dx and back, so a feature thinner than about two grid cells is averaged with its neighbours on the way through and comes back as the average. The quantity that decides whether a hard shell is possible is therefore the shell's thickness *in MPM grid cells*, and it had never been measured:

object	shell thickness	in fine cells	in MPM grid cells	a stiffness contrast is
orange	0.01910	3.2	2.00	exactly at the threshold
watermelon	0.02571	3.0	1.25	not resolvable, needs 4.8 cells
doughnut	0.01234	3.0	0.87	not resolvable, needs 6.8 cells

4.3.3 Shell thickness as a parameter of route 2

That threshold is what route 2 can act on. It builds the lattice from a field, so the skin band β is a number passed to the builder rather than a property of someone else's model. The prediction above says the doughnut needs about seven fine cells. Building it at four settings:

β, fine cells	cells	measured thickness	in MPM grid cells	
3 (the default)	1,129,008	4.1	1.17	no
5	1,486,904	5.5	1.57	no
7	1,827,832 (+62%)	7.2	**2.06**	**yes**
9	2,148,936 (+90%)	9.0	2.59	yes

4.3.4 Solver response across the threshold

The arithmetic says a shell under two grid cells cannot hold a stiffness contrast. Simulating both settings says whether that is what happens. The object falls under its own weight with the material field applied, and the solver reports each body's extent along the gravity axis as a fraction of its rest extent, separately for the cells the field called skin and the ones it called interior. A hard shell keeps its own extent and lets the interior lose more, so the sign of the gap between them is the question, not its size.

doughnut, at deepest compression	shell, in grid cells	skin extent	interior extent	interior − skin
β = 3, falling	1.17	0.9220	0.9259	**+0.0040**
β = 7, falling	2.06	0.9334	0.9318	**−0.0016**
β = 3, falling and cut	1.17	0.9264	0.9333	**+0.0070**
β = 7, falling and cut	2.06	0.9280	0.9177	**−0.0103**

Three things about that field. It is not equation (14): it is the weak two-rule field of `material_field.py`, level deciding skin from interior and lightness grading the interior, which is the version a learned decomposition has to beat rather than the clustered one equation (14) defines. Its numbers are the `orange` preset, 8.0e6 against 6.0e5, applied to the doughnut. And the interior is graded rather than constant, spanning 6.0e5 to 6.0e6, so the contrast between the shell and the stiffest interior is 1.33× and not the 13× a shell-against-softest reading gives. A 1.33× contrast is a weak thing to look for a threshold with, and it is part of why the effect is small. The 6.0e6 top of that interior range is what the grading rule can reach rather than what it was measured to reach, so 1.33× is a bound and not a measurement.

Two confounds are not controlled. The same preset makes the skin *lighter* than the interior, 750 against 950 in density, so an extent measured under gravity is being pushed by a density contrast in the opposite direction to the stiffness one and the paper cannot separate them. And the β = 7 build is not the β = 3 build with a thicker skin: it has 62% more cells and a much larger skin fraction, so at 8.0e6 in the shell the whole body is stiffer, and both extents rise between the two settings, which is the signature of a stiffer object rather than of a resolved shell. The control that separates these is the uniform preset at both β, which the material code ships for exactly this purpose. It has now been run, with the level labels present so that the same per-level extent is recorded, and only the stiffness made uniform.

doughnut, extent gap (interior − skin) at the last frame	β = 3	β = 7
uniform stiffness, labels present	**+0.0630**	**+0.0628**
the material field applied	+0.0288	+0.0515
difference the field makes	−0.0342	−0.0113

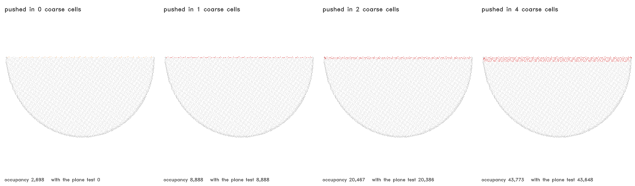
The control does what a control should: it shows that the geometry alone produces a gap, that the gap is the same at both band widths to within a thousandth, and therefore that a raw gap is not evidence about a shell. Against that baseline the material field does move the result, so the field is reaching the solver. But it moves it *more* at β = 3 than at β = 7, which is the opposite ordering to the one a shell becoming resolvable would produce, and it is consistent with the τ_s values above: on the grid the solver integrates, neither build is near the threshold, so there is no reason to expect the thicker band to behave differently in kind. **We report this as a negative result.** The band width is a parameter route 2 can set, the criterion for choosing it is stated and computable, and the experiment intended to confirm the criterion does not confirm it.

The four gaps are 0.16%, 0.40%, 0.70% and 1.03% of the object's extent, so the effect is a fraction of a percent at three of the four settings and reaches 1% at one. Each cell is a single deterministic run: there is no repeat, no seed variation and therefore no noise floor to compare the sign flip against. It is detectable in the solver's report and not in a render. What it establishes is that the diagnosis predicted a threshold and that crossing it reverses the sign of the response, on one object. Whether a contrast large enough to *look* like a peel needs a thicker shell again, a finer solver grid or a different transfer is not answered here.

4.3.5 Collision

Equation (13) is two tests, and the second is the one that earns its place. At rest, with the two halves exactly as the cut left them, occupancy alone reports 2,698 particles of the orange inside the other piece, 2,699 of the watermelon and 943 of the doughnut; the sign test takes all three to zero.

That zero is a consequence rather than a finding, and it is worth writing down as one. A leaf assigned to piece B has its centre on B's side, so a point on A's side that falls inside a B leaf must lie within the leaf's half-diagonal of the plane, which is the bound of equation (9), and the sign test removes exactly that band. The at-rest column is therefore a check that the indexing has no aliasing bug, not evidence about contact. The evidence is the push column: penetration has to grow with separation and nothing in the construction forces it to.



A slab through the contact region of the cut orange, every particle drawn as what the test says about it: grey for outside the other piece, orange for claimed by occupancy alone, red for still claimed once the sign test against Π is applied. Which piece a particle belongs to is decided by the plane and not by the leaf it sits in, because asking whether a particle labelled by its own leaf falls inside a different leaf is circular and answers zero against zero. At rest the orange band is the leaves the cut straddles, 2,698 particles wide, and no particle survives the sign test. Pushing one piece into the other turns the band red and grows it in proportion, which is what makes this a contact test rather than a coincidence.

object	particles	solid cells	at rest, occupancy alone	with the plane test	pushed 1 / 2 / 4 cells in
orange	1,231,312	894,304	2,698 of 610,723	**0**	8,888 / 20,386 / 43,648
watermelon	1,065,856	748,248	2,699 of 527,791	**0**	8,978 / 20,373 / 43,781
doughnut	942,840	567,448	943 of 473,192	**0**	2,722 / 6,542 / 13,994

The last column is the check that the test is a contact test and not a coincidence: pushing one piece into the other by one, two and four coarse cells has to report more penetration each time, and it does, on every object.

4.3.6 Surface binding

Collision moves pieces; binding is what carries the visible surface with them. Equation (16) fixes each surface vertex to its own piece's nearest particles at bind time, so the surface follows whatever the solver does to them, including deformations no rigid transform can express.

One piece under a non-affine deformation, its surface bound to the particles, and the same particles driving it two ways: a per-piece rigid fit on the left, which is the best a single transform can do, and the particle binding on the right, both coloured by how far each vertex has left its material on one shared scale. The rigid panel goes red and stays undeformed; the bound panel bends and stays at 0.00. The bind residual is 0.34 to 0.45 of a fine cell in the mean and 1.02 to 1.23 at the 95th, over 480,344, 419,404 and 442,956 surface vertices carried on 1.23M, 1.07M and 0.94M particles. The surface is the whole O-Voxel exterior now rather than a cut patch, so what is bound is every visible vertex the object has.

4.4 Appearance

4.5 Correctness, storage and runtime

4.5.2 The cut face

This is the paper's first claim and the one a viewer sees. Each crossed cell contributes one convex polygon, obtained by intersecting the plane with the cell's twelve edges in closed form. No primitive is asked to approximate a plane, so there is no error to tune away and the only departure in a cut area is the voxelisation of the silhouette. A method that draws a cut face from whatever primitives lie near the plane has neither property, and cannot state either.

ball, radius in cells	plane	cut area	π r^2	area error	worst vertex off the plane
6	axis-aligned, d=0	112.0	113.1	0.97%	0.00e+00
12	axis-aligned, d=0	448.0	452.4	0.97%	0.00e+00
24	axis-aligned, d=0	1804.0	1809.6	0.31%	0.00e+00
48	axis-aligned, d=0	7232.0	7238.2	**0.09%**	0.00e+00
6	oblique, off-grid	115.6	112.7	2.58%	4.44e−16
12	oblique, off-grid	451.0	452.0	0.22%	6.66e−16
24	oblique, off-grid	1803.4	1809.1	0.32%	1.78e−15
48	oblique, off-grid	7233.7	7237.8	**0.06%**	3.55e−15
torus, R=14, r=5	across the axis	868.0	4π Rr = 879.6	1.3%	0.00e+00

This is the decomposition demonstrated rather than asserted. The area error falls roughly as h with resolution, from about 1% at six cells to 0.06 to 0.09% at forty-eight, which is what a voxelised silhouette should do and what a plane approximated by primitives would not. The residual of the plane itself does not fall and does not need to: it is 0.00e+00 for an axis-aligned plane through the origin, where the intersection lands on a coordinate exactly, and a few multiples of the machine epsilon for an oblique off-grid plane at every resolution. The cut face carries no error term that resolution has to chase; the silhouette does.

4.5.6 Storage, static and during a cut

The other half of the memory line, and the one the coarse interior exists for: storage should be proportional to hidden spatial complexity, with the high-resolution cost paid only where a cut has made something visible and returned when it has not. Counted as fields rather than as process memory: a Gaussian stores position, scale, rotation, opacity and colour; a cube cell stores a feature, a level and its membership, with the position implicit in the address.

Three accountings, and they answer different questions. Holding the count fixed and changing only the per-element fields is 1.6×. Paying for the integer coordinates a sparse lattice has to store, which every structure built over it keys on, takes that to 1.2×. Adding the dual-grid exterior, which the method needs and a Gaussian model carries inside its own primitives, takes the orange past parity: 55.6 MiB against 46.9 for route 1. The saving against the published models is therefore almost entirely in *how many things there are*, not in what each one costs, and on an object whose lattice is mostly fine cells it can vanish. None of these figures is measured against a compressed Gaussian model, which would move them again.

orange	as Gaussians	as cube cells	a cut, while live	folded back
route 1, 877,494 cells	46.9 MiB	28.5 MiB (1.6×) / **38.5 with the index** / 55.6 with the exterior	+98,714 leaves, 1.6 MiB (4.2%)	exactly returned
route 2, 1,231,312 cells	65.8 MiB	39.9 MiB (1.6×) / **54.0 with the index** / 71.7 with the exterior	+97,209 leaves, 1.6 MiB (2.9%)	exactly returned

5 Conclusion

Separating the two jobs costs less than it sounds and gives back more than the rendering. The hidden volume becomes occupancy plus a low-dimensional feature, with no σ to tune against h; the visible boundary becomes a surface with an explicit polygonal extent, which is what lets a flat cut face be flat rather than a plane approximated by ellipsoids. Cutting stays discrete where discreteness is cheap and becomes analytic exactly where a staircase would show. And the lattice's own advantages survive intact: connectivity, piece identity and collision are integer operations on a structured grid, not queries against arbitrary free particles.

Against the two methods that fill an interior with Gaussians, the difference is what a volume costs. On the same watermelon the interior here is 1,065,856 cells generated or 1,741,169 quantised, against 7,959,789 and 1,381,100 primitives, and counting the sparse index and the exterior surface the method actually needs, 62.6 or 163.5 MiB against 425.1 and 310.8. Swept over four decades of ρ the trade is real in both directions, holes below 0.1 and blending above 0.25, and both published models sit far to the left of that window and buy their coverage with count instead.

Coverage is where the revision changed the answer. At 256 px nothing separates the four representations, and by 2048 px GaussianFluent's section is 38.7% background, FruitNinja's 2.4% and this work's own splatted lattice's 3.3%, while the same lattice drawn as a cube volume holds 0.07%. That is one object at three image sizes and it is the claim we would defend; the 20.5% and 46.7% this page once reported came from drawing each primitive at radius σ/2, which is a fair question about what a primitive covers and the wrong instrument for what a renderer draws.

Appearance does not pay for that, and the appearance numbers do not all go one way. On the orange, every arm here beats FruitNinja's released model, and the arm that beats it by the most longitudinally saw no photograph of the fruit. On the watermelon, GaussianFluent's released model takes the best FID in the table at 170.2 against our 180.9 and FruitNinja's 182.0, on a different fruit and through cameras we had to choose for it. Every one of those numbers sits inside a reference set that scores 46.3 and 60.4 against itself, which is the reading we would defend: at this sample size these interiors are not separable by the metrics the task has, and what separates the representations is what they cost and what they can state about a cut face.

Where the metrics do separate something is dependence. Removing object-specific supervision from this work moves it 1.4 FID at fixed code and budget. FruitNinja's pipeline run from its own code with a stock checkpoint scores 378.7 against its released 102.2; that gap is an upper bound, since the two runs differ in training budget as well, but the collapse is visible in the render before any metric is computed. GaussianFluent needs no fine-tuned model either, and it fractures by continuum damage where

this work separates by connected components, so fracture is a capability both have rather than an axis between them. What is left between the three is the volume itself: a cube tiles space and a sub-pixel Gaussian does not, and that is measured in section 4.2.2 on terms none of the three chose.

References

1. B. Kerbl, G. Kopanas, T. Leimkühler and G. Drettakis. *3D Gaussian Splatting for Real-Time Radiance Field Rendering.* ACM TOG 42(4), 2023.
2. B. Mildenhall et al. *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis.* ECCV 2020.
3. F. Wu and Y. Chen. *FruitNinja: 3D Object Interior Texture Generation with Gaussian Splatting.* arXiv:2411.12089, CVPR 2025.
4. W. E. Lorensen and H. E. Cline. *Marching Cubes: A High Resolution 3D Surface Construction Algorithm.* SIGGRAPH 1987.
5. T. Ju, F. Losasso, S. Schaefer and J. Warren. *Dual Contouring of Hermite Data.* ACM TOG 21(3), SIGGRAPH 2002.
6. S. Schaefer and J. Warren. *Dual Marching Cubes: Primal Contouring of the Dual Grid.* Pacific Graphics 2004.
7. L. P. Kobbelt, M. Botsch, U. Schwanecke and H.-P. Seidel. *Feature Sensitive Surface Extraction from Volume Data.* SIGGRAPH 2001.
8. M. Garland and P. S. Heckbert. *Surface Simplification Using Quadric Error Metrics.* SIGGRAPH 1997.
9. T. Ju. *Robust Repair of Polygonal Models.* ACM TOG 23(3), SIGGRAPH 2004.
10. J. Xiang et al. *Native and Compact Structured Latents for 3D Generation.* CVPR 2026. TRELLIS.2.
11. J. Xiang et al. *Structured 3D Latents for Scalable and Versatile 3D Generation.* CVPR 2025.
12. T. Lu et al. *Scaffold-GS: Structured 3D Gaussians for View-Adaptive Rendering.* CVPR 2024.
13. B. Zhang et al. *GaussianCube: A Structured and Explicit Radiance Representation for 3D Generative Modeling.* NeurIPS 2024.
14. K. Ren et al. *Octree-GS: Towards Consistent Real-time Rendering with LOD-Structured 3D Gaussians.* IEEE TPAMI, 2025.
15. A. Yu, R. Li, M. Tancik, H. Li, R. Ng and A. Kanazawa. *PlenOctrees for Real-time Rendering of Neural Radiance Fields.* ICCV 2021.
16. C. Sun, M. Sun and H.-T. Chen. *Direct Voxel Grid Optimization: Super-fast Convergence for Radiance Fields Reconstruction.* CVPR 2022.
17. T. Müller, A. Evans, C. Schied and A. Keller. *Instant Neural Graphics Primitives with a Multiresolution Hash Encoding.* ACM TOG 41(4), 2022.
18. T. Xie et al. *PhysGaussian: Physics-Integrated 3D Gaussians for Generative Dynamics.* arXiv:2311.12198, CVPR 2024.
19. C. Jiang, C. Schroeder, J. Teran, A. Stomakhin and A. Selle. *The Material Point Method for Simulating Continuum Materials.* SIGGRAPH Courses 2016.
20. A. Stomakhin, C. Schroeder, L. Chai, J. Teran and A. Selle. *A Material Point Method for Snow Simulation.* ACM TOG 32(4), 2013.
21. Y. Hu et al. *A Moving Least Squares Material Point Method with Displacement Discontinuity and Two-Way Rigid Body Coupling.* ACM TOG 37(4), 2018.
22. Y. Jiang et al. *VR-GS: A Physical Dynamics-Aware Interactive Gaussian Splatting System in Virtual Reality.* SIGGRAPH 2024.
23. B. Huang, Y. Chen, R. Lu, G. Zeng, H. Zha, Y. Pei and S. Huang. *GaussianFluent: Gaussian Simulation for Dynamic Scenes with Mixed Materials.* arXiv:2601.09265, CVPR 2026.
24. J. P. Lewis, M. Cordner and N. Fong. *Pose Space Deformation: A Unified Approach to Shape Interpolation and Skeleton-Driven Deformation.* SIGGRAPH 2000.
25. R. W. Sumner, J. Schmid and M. Pauly. *Embedded Deformation for Shape Manipulation.* ACM TOG 26(3), 2007.
26. R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer. *High-Resolution Image Synthesis with Latent Diffusion Models.* CVPR 2022.
27. B. Poole, A. Jain, J. T. Barron and B. Mildenhall. *DreamFusion: Text-to-3D using 2D Diffusion.* ICLR 2023.
28. E. Richardson et al. *TEXTure: Text-Guided Texturing of 3D Shapes.* SIGGRAPH 2023.
29. J. Kopf et al. *Solid Texture Synthesis from 2D Exemplars.* ACM TOG 26(3), 2007.
30. H. Liu et al. *InFusion: Inpainting 3D Gaussians via Learning Depth Completion from Diffusion Prior.* arXiv:2404.11613, 2024.
31. E. Liang et al. *InnerGS: Internal Scenes Reconstruction and Segmentation with Gaussian Splatting.* arXiv, 2025.
32. M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter. *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium.* NeurIPS 2017. (FID)
33. M. Bińkowski, D. J. Sutherland, M. Arbel and A. Gretton. *Demystifying MMD GANs.* ICLR 2018. (KID)
34. J. Hessel, A. Holtzman, M. Forbes, R. Le Bras and Y. Choi. *CLIPScore: A Reference-free Evaluation Metric for Image Captioning.* EMNLP 2021.
35. M. J. Chong and D. Forsyth. *Effectively Unbiased FID and Inception Score and Where to Find Them.* CVPR 2020. (why FID is unusable at the sample sizes here)
36. T. Kynkäänniemi, T. Karras, M. Aittala, T. Aila and J. Lehtinen. *The Role of ImageNet Classes in Fréchet Inception Distance.* ICLR 2023.
37. P. Debevec, C. Taylor and J. Malik. *Modeling and Rendering Architecture from Photographs.* SIGGRAPH 1996.
38. C. Buehler, M. Bosse, L. McMillan, S. Gortler and M. Cohen. *Unstructured Lumigraph Rendering.* SIGGRAPH 2001.
39. C. Ericson. *Real-Time Collision Detection.* Morgan Kaufmann, 2004, §5.2.3.
40. S. Niedermayr, J. Stumpfegger and R. Westermann. *Compressed 3D Gaussian Splatting for Accelerated Novel View Synthesis.* CVPR 2024.
41. J. C. Lee, D. Rho, X. Sun, J. H. Ko and E. Park. *Compact 3D Gaussian Representation for Radiance Field.* CVPR 2024.
42. Z. Fan et al. *LightGaussian: Unbounded 3D Gaussian Compression with 15× Reduction.* NeurIPS 2024.
43. Y. Chen et al. *HAC: Hash-grid Assisted Context for 3D Gaussian Splatting Compression.* ECCV 2024.
44. W. Morgenstern et al. *Compact 3D Scene Representation via Self-Organizing Gaussian Grids.* ECCV 2024.
45. M. Bagdasarian et al. *3DGS.zip: A Survey on 3D Gaussian Splatting Compression.* Eurographics STAR 2025.
46. G. Fang and B. Wang. *Mini-Splatting: Representing Scenes with a Constrained Number of Gaussians.* ECCV 2024.
47. S. Mallick et al. *Taming 3DGS: High-Quality Radiance Fields with Limited Resources.* SIGGRAPH Asia 2024.
48. N. Molino, Z. Bao and R. Fedkiw. *A Virtual Node Algorithm for Changing Mesh Topology During Simulation.* SIGGRAPH 2004.
49. E. Sifakis, K. Der and R. Fedkiw. *Arbitrary Cutting of Deformable Tetrahedra.* SCA 2007.
50. D. Koschier, J. Bender and N. Thuerey. *Robust eXtended Finite Elements for Complex Cutting of Deformables.* ACM TOG 36(4), SIGGRAPH 2017.
51. D. Bielser and M. Gross. *Interactive Simulation of Surgical Cuts.* Pacific Graphics 2000.
52. J. Wolper, Y. Fang, M. Li, J. Lu, M. Gao and C. Jiang. *CD-MPM: Continuum Damage Material Point Methods for Dynamic Fracture Animation.* ACM TOG 38(4), SIGGRAPH 2019.
53. Z. Yu, A. Chen, B. Huang, T. Sattler and A. Geiger. *Mip-Splatting: Alias-free 3D Gaussian Splatting.* CVPR 2024.
54. Z. Liang et al. *Analytic-Splatting: Anti-Aliased 3D Gaussian Splatting via Analytic Integration.* ECCV 2024.
55. J. T. Barron et al. *Mip-NeRF: A Multiscale Representation for Anti-Aliasing Neural Radiance Fields.* ICCV 2021.

The two references that carry the most weight here are [5] and [10]: dual contouring is where the idea of one vertex per cell placed by a quadric comes from, and TRELLIS.2's Flexible Dual Grid is the implementation this work converts into. The lattice

this work starts from, and its numbers are the baseline throughout.